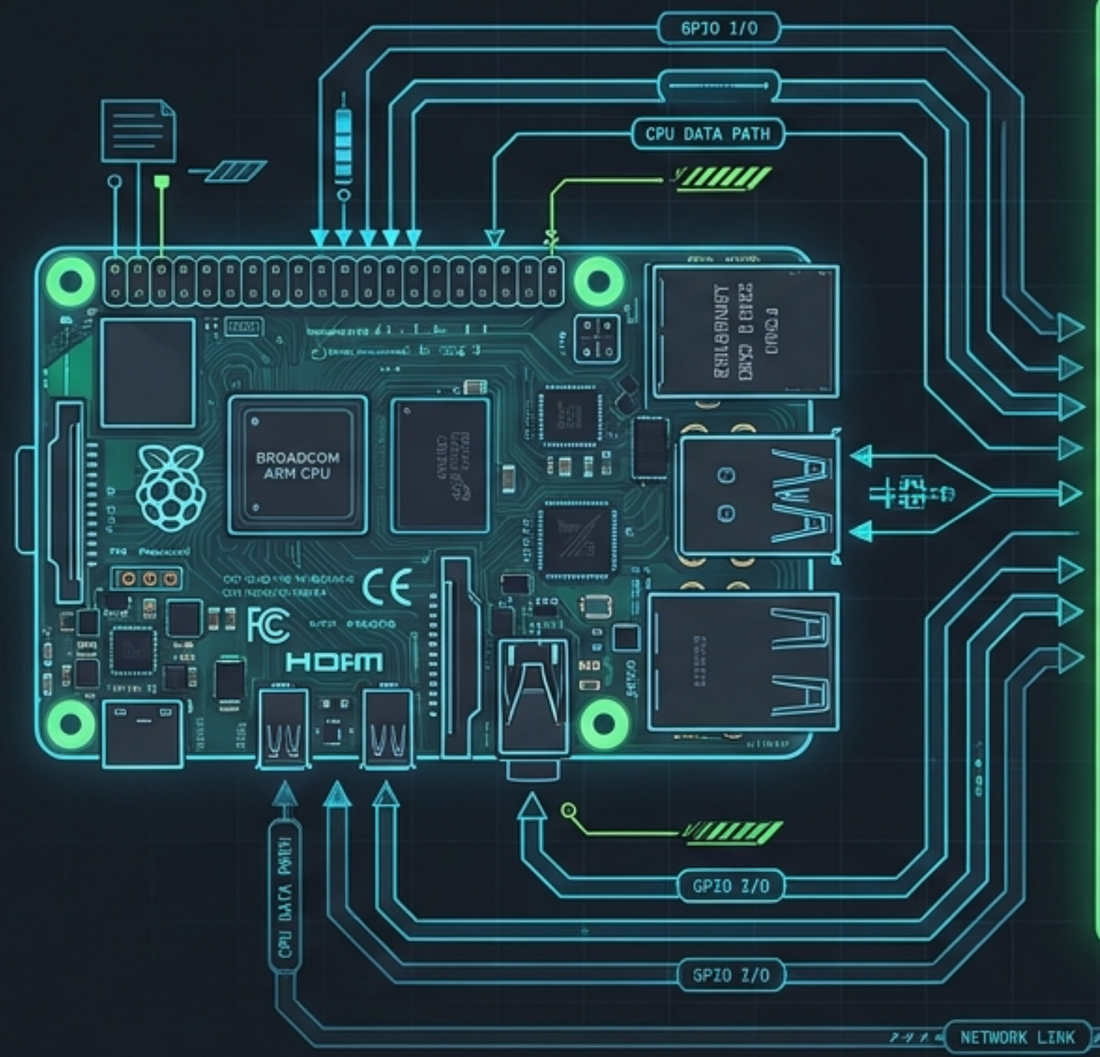


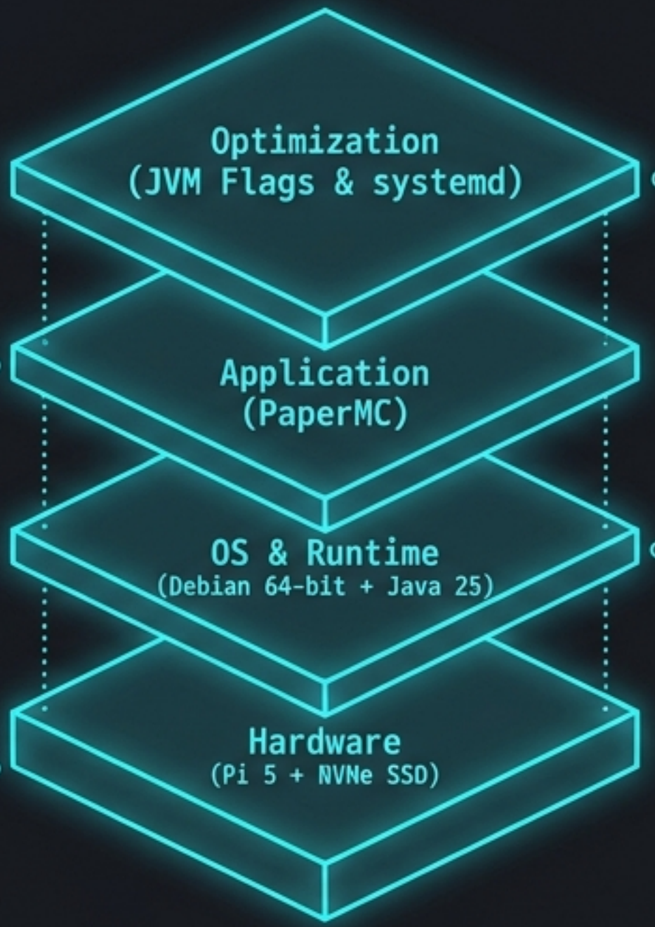

# Minecraft Server auf Raspberry Pi: Das DevOps-Runbook

Ein architektonischer Blueprint für maximale Performance – Fokus auf Java 25, PaperMC API & ARM-optimierte Start-Parameter.

Gemini Notebook

# Die Server-Architektur: Von Bare Metal bis zur Laufzeitumgebung

```
pi@server:~$ java -version
openjdk version '25-ea'
openjdk linux (svarloo/2.6.15.2)
pi@server:~$
```




```
JVM Args: -Xms4G
          -Xmx6G
          +UseZGC +ZGenerational
Systemd: CPUAffinity=... Nice=-20
Kernel: transparent_hugepage=always
```

```
Server Software: PaperMC 1.21+
Engine: High-Performance Minecraft
Plugins: Minimal/Optimized
Config: tuned paper-global.yml & bukkit.yml
```

```
OS: Debian Bookworm aarch64
Kernel: Linux 6.x
JDK: OpenJDK 25 (Early Access)
GC: ZGC/Shenandoah
```

Internet -> [Firewall] -> Reverse Proxy (Nginx) -> [PaperMC:25565]

```
Architecture: ARM64 (BCM2712)
Storage: NVMe M.2 via PCIe Gen 2/3
RAM: 8GB LPDDR4X-4266
```

```
pi@server:~$ top
top - 22:27:0  up 56 top,  1 user,  load processes: 3.05, 0.66
Tasks: 260 total,   2 running, 25 sleeping,   0 stopped,   0 ze
%Cpu(s): 12.6 us,  0.0 sy,  0.9 ni 36.5 id,  0.0 wa,  0.0 hi,
KiB Mem :  265309 total,    25905 free,    30008 used,   2303
KiB Swap:       0 total,        0 free,        0 used.   2303

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+
 3926 server    20   0   40026   2900   3296 S   5.8   2.2   2:39.3
 1394 server    20   0   15098   1350   1230 S   0.0   2.2   2:20.8
    1 root      20   0  223926   2909   2220 S   0.0   3.0   5:30.9
    2 root      20   0       0      0      0 S   0.0   0.0   0:00.0
    3 root      20 -20       0      0      0 S   0.0   0.0   0:00.0
    4 root       0 -20       0      0      0 S   0.0   0.0   0:00.0
    5 root      20 -20       0      0      0 S   0.0   0.0   0:00.0
```

Hohe TPS (Ticks Per Second) auf ARM-Hardware entstehen nicht durch Zufall, sondern durch das perfekte Zusammenspiel von optimierter Software und hardwarenahem Tuning.

Gemini Notebook

# Diagnose-Matrix: Hardware-Realität & Software-Wahl

## Hardware (Pi 4 vs. Pi 5)

Pi 4 (Min. 4GB, max 2-4 Spieler).

Pi 5 (Empfohlen 8GB, 8-12 Spieler, 2-3x schnellere CPU).

## Betriebssystem

64-bit OS (Raspberry Pi OS Lite oder Ubuntu Server) ist ein absolutes Muss für RAM-Zuweisungen > 3GB.

## Speicher (SD vs. SSD)

SD-Karten: Hohes Korruptionsrisiko durch I/O-Last.
NVMe/USB 3.0 SSD: Zwingend erforderlich für Chunk-Loading.

## Edition (Java vs. Bedrock)

Bedrock benötigt x86-Emulation (Box64) auf ARM und ist extrem instabil.
Die Lösung: Java Edition + PaperMC.

Gemini Notebook

# Runtime-Ebene: Installation von Java 25




Debian 13 / Ubuntu 24.04+
(Native Paketquellen)

```
sudo apt install -y openjdk-25-jre-headless
```

Debian 12 - älteres Pi OS
(Temurin via Adoptium)

```
wget -O -  https://packages.adoptium.net/
artifactory/api/gpg/key/public |
sudo gpg --dearmor -o
/etc/apt/trusted.gpg.d/adoptium.gpg\
sudo apt install -y temurin-25-jdk
```

**Warnung:** Aktuelle PaperMC-Builds (1.21+) setzen **zwingend Java 21 oder 25** voraus. Java 17 führt zu Abstürzen.

Gemini Notebook

# Das Software-Dilemma: Warum Vanilla auf ARM scheitert



### Vanilla Minecraft

Single-threaded Welt-Generierung, hohe CPU-Last, ineffizientes Entity-Management.



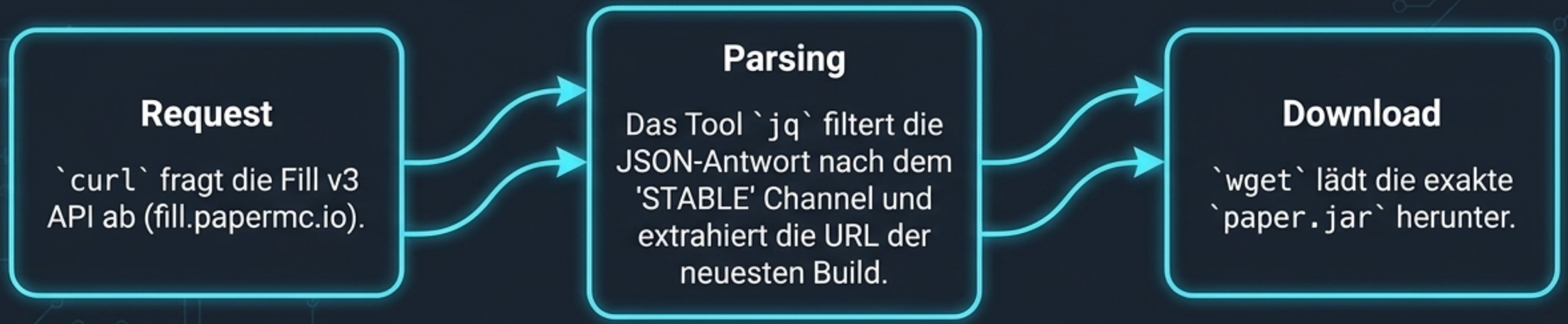
### PaperMC

Asynchrones Chunk-Loading, 30–50% weniger CPU-Last, optimierte Garbage Collection, API-Support für Plugins.

⚠ **Fazit: PaperMC ist auf ressourcenbeschränkter Hardware (wie dem Pi) keine Option, sondern eine absolute Notwendigkeit.**

Gemini Notebook

# Automatisierung: PaperMC via API beziehen




```
VERSION=26.3
URL=$(curl -s https://fill.papermc.io/v3/projects/paper/versions/$VERSION/builds | jq -r
'first(.[] | select(.channel == "STABLE")) | .downloads."server:default".url')
wget -O paper.jar "$URL"
```

Gemini Notebook

# Initialisierung & EULA-Bestätigung

**Run**

```
java -jar paper.jar --nogui
```

**Crash & Generate**

Server stoppt sofort (gewollt). Generiert die grundlegende Dateistruktur und `eula.txt`.

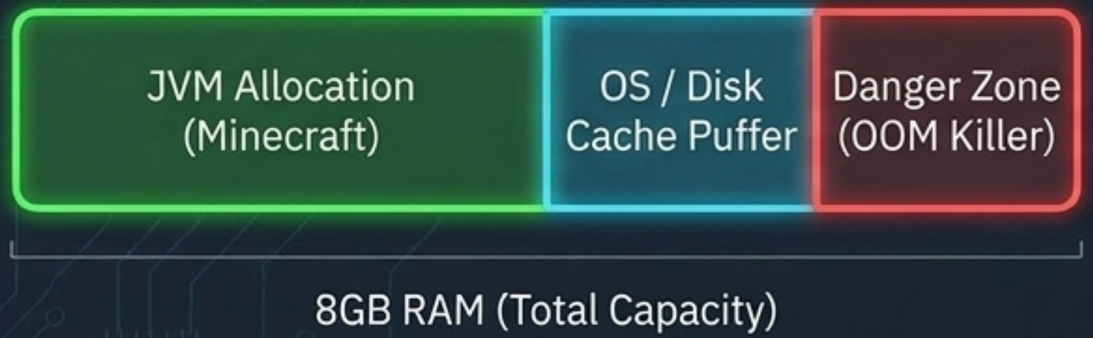
**Accept**

```
sed -i 's/eula=false/eula=true/' eula.txt
```

Dieser Schritt generiert die grundlegende Dateistruktur für die weiteren Tuning-Maßnahmen.

Gemini Notebook

# Engine-Tuning Teil 1: RAM-Zuweisung & Systemstabilität




### Das Limit

Niemals 100% des RAMs zuweisen. Das OS und der Festplatten-Cache benötigen Puffer. Überbelegung triggert den Linux **Out Of Memory Killer**.

### Die Parameter

Die Parameter: “`-Xms`” (Start-RAM) und “`-Xmx`” (Maximal-RAM). Gleiche Werte verhindern Ruckler durch Heap-Resizing.

### Hardware-Empfehlungen

Pi 5 (8GB): “`-Xms4G -Xmx4G`” (Der Sweet-Spot)
Pi 4 (4GB): “`-Xms2G -Xmx2G`”

Gemini Notebook

# Engine-Tuning Teil 2: Garbage Collection (Aikar’s Flags für ARM)




**Der Kern.** Aktiviert den Garbage Collector, der am besten mit begrenztem Speicher (ARM) skaliert.

**Speicher-Blöcke.** Verhindert Speicher-Fragmentierung und optimiert die Region-Größe für den Pi.

**Lag-Vermeidung.** Zwingt Java, Speicherbereinigungen kurz zu halten, um spürbare Lag-Spikes im Spiel zu verhindern.

Gemini Notebook

# Synthese: Das vollständige Start-Script (start.sh)

start.sh

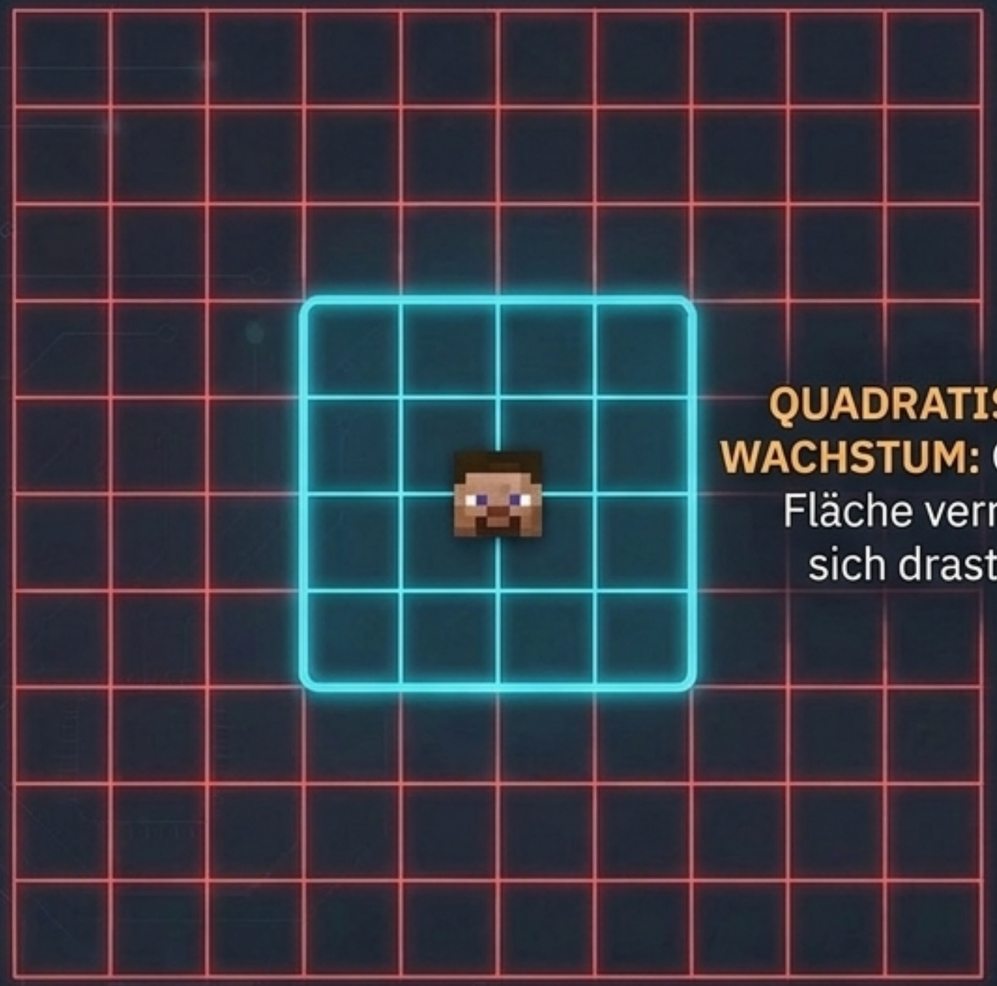
```
#!/bin/bash

java \
  -Xms2G -Xmx4G \
  -XX:+UseG1GC \
  -XX:+ParallelRefProcEnabled \
  -XX:MaxGCPauseMillis=200 \
  -XX:+UnlockExperimentalVMOptions \
  -XX:+DisableExplicitGC \
  -XX:G1NewSizePercent=30 \
  -XX:G1MaxNewSizePercent=40 \
  -XX:G1ReservePercent=20 \
  -XX:InitiatingHeapOccupancyPercent=15 \
  -jar paper.jar --nogui
```

Ausführbar machen nicht vergessen: `chmod +x start.sh`

Gemini Notebook

# Welt-Konfiguration: Die Mathematik der Chunks (server.properties)



**QUADRATISCHES WACHSTUM:** Geladene Fläche verringert sich drastisch!

`view-distance=6 bis 8`

Standard ist 10. Die Reduzierung von 10 auf 6 senkt die geladene Fläche um ca. 64%. Dies ist der effektivste Hebel für die Pi-Performance.

`simulation-distance=4 bis 6`

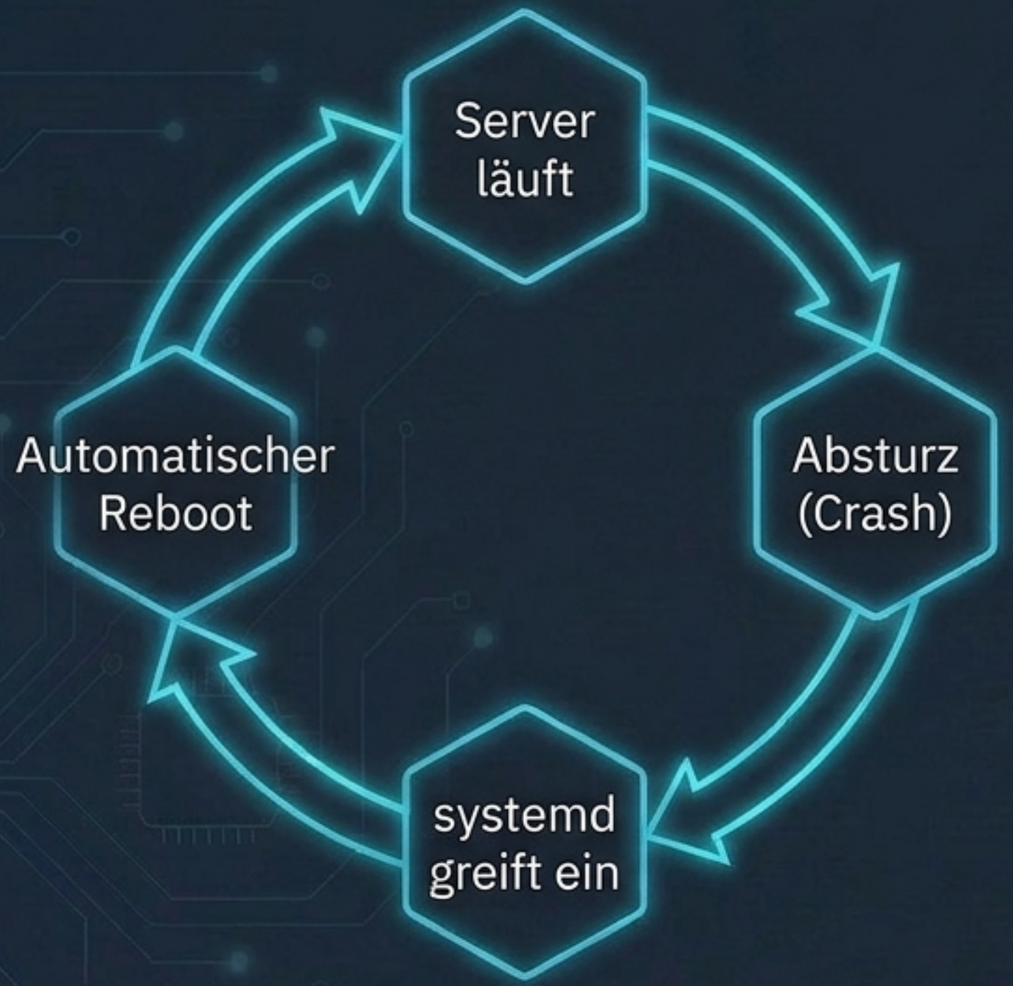
Limitiert, in welchem Radius Redstone und Entities (Mobs) vom Server aktiv berechnet werden.

`max-players=8`

Realistisches und stabiles Limit für ARM-Hardware (Pi 5).

Gemini Notebook

# Autostart-Architektur: Resilienz durch systemd




**Niemals** den Server in einer aktiven Terminal-Session (ohne screen/tmux) laufen lassen.

**/etc/systemd/system/minecraft.service**

```
User=minecraft (Sicherheits-Best-Practice)
ExecStart=/opt/minecraft/server/start.sh
Restart=on-failure (Automatischer Reboot bei Absturz)
```

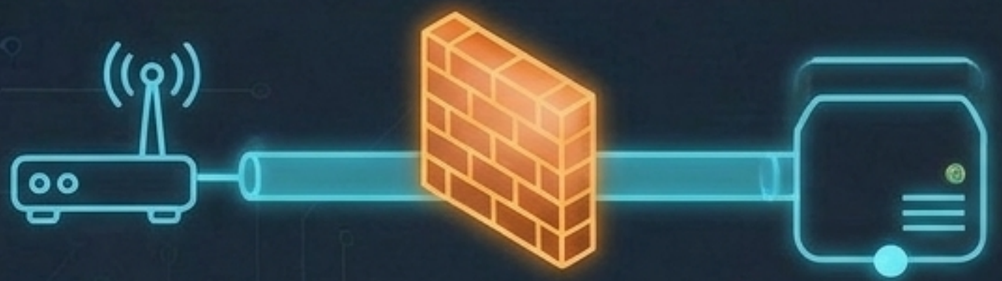
**Aktivierung**

```
sudo systemctl enable minecraft.service
```

Gemini Notebook

# Netzwerk & Sicherheit: Routing-Optionen

## Option A: Klassisch



UFW & Port Forwarding

```
sudo ufw allow 25565/tcp
```

Erfordert offene Ports am Router und einen DynDNS-Dienst.

## Option B: Modern & Sicher



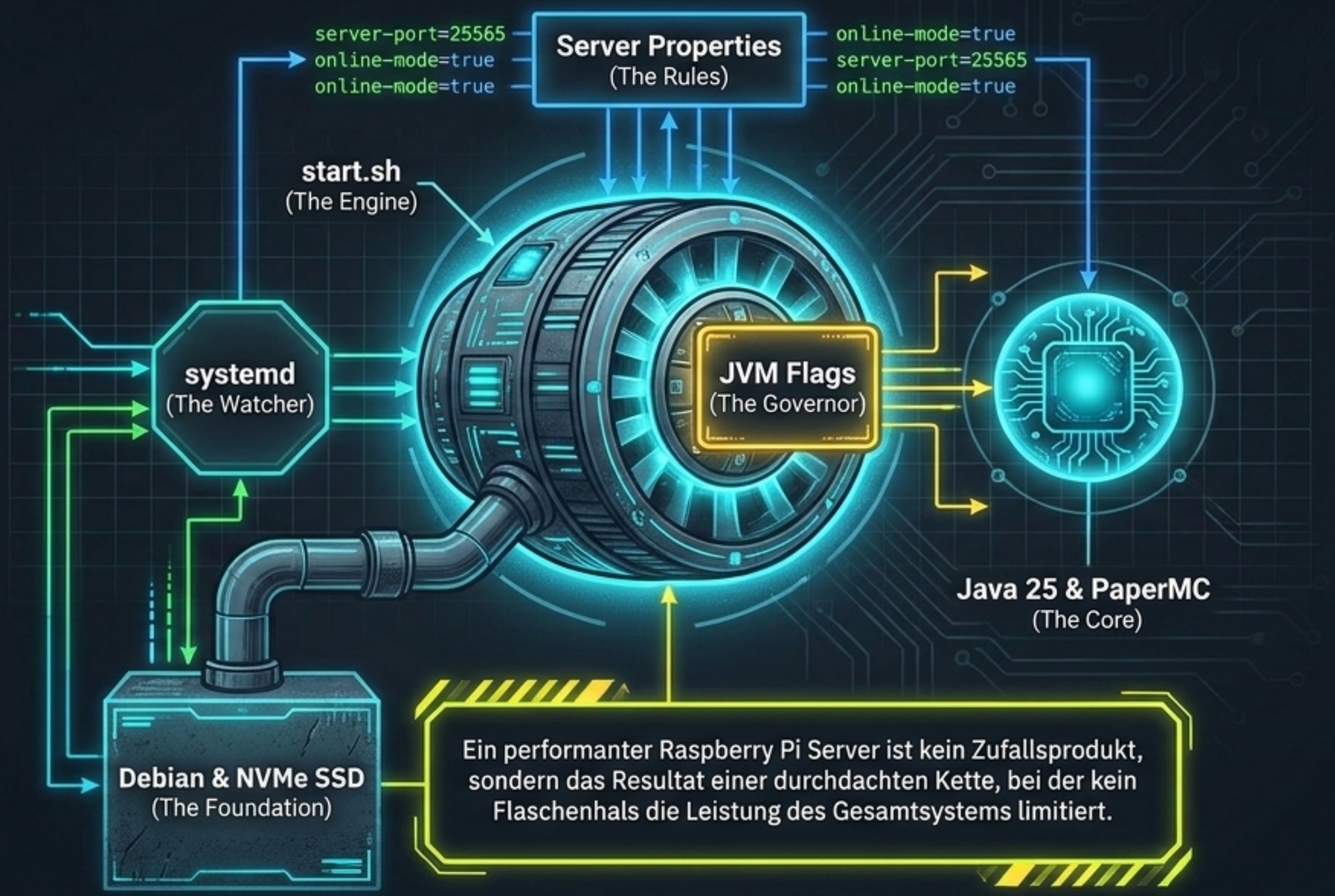

Keine offenen Ports am Heimrouter nötig.

Spieler verbinden sich in ein privates Mesh-Netzwerk.

Verbindung im Spiel über die zugewiesene 100.x.y.z IP-Adresse.

Gemini Notebook

# System-Synthese: Das finale Architektur-Modell




Gemini Notebook

# Wartung & Operations (Cheat Sheet)

## Performance prüfen

In-Game `/tps` eingeben (Ziel: 20.0, Lags ab <18).

Temperatur: `vcgencmd measure_temp` (⚠ < 80°C).

## Service steuern

```
sudo systemctl restart minecraft
sudo systemctl stop minecraft
sudo systemctl status minecraft
```

## Logs auslesen

Echtzeit-Serverkonsole betrachten:

```
sudo journalctl -u minecraft -f
```

## Welt Pre-Generation

Plugins wie Chunky nutzen, um die Welt nachts vorab zu generieren. Verhindert drastisch Lag-Spikes bei der Erkundung.

Gemini Notebook